Binary Classification

A neural network that predicts whether a customer leaves, trained on a table of 4960 labelled customers and measured on customers it was not trained on.

---

01 Preprocessing fitted on the training split
02 Splits and class balance
03 Logistic regression and an MLP
04 Logits and BCEWithLogitsLoss
05 The training loop
06 Dataset and DataLoader
07 Evaluation
08 Validation and early stopping
09 Saving, loading and the GPU

Scope. Arrays, masks and broadcasting, classes with `__init__` and methods, tensors, autograd, `nn.Module` and a network trained by gradient descent are assumed.

After reading it you should be able to do the following without looking anything up.

- Build the feature matrix with every statistic taken from the training rows only.
- Split the rows into training, validation and test sets and balance the two classes.
- Define logistic regression and an MLP and count their parameters.
- Compute the binary cross-entropy from logits and explain why the loss takes logits.
- Write the training loop over minibatches and epochs, by hand and with a `DataLoader`.
- Compare an accuracy with its baseline, stop training early, save a model and train on a GPU.

Every example assumes `import copy`, `import csv`, `import numpy as np`, `import matplotlib.pyplot as plt`, `import torch` and `import torch.nn as nn`. Code comments give shapes, as `(going in) -> (coming out)`. Values in `# ->` comments are rounded to the digits shown.

01

Preprocessing fitted on the training split

A network needs its input as one matrix of floats with no missing values, but our CSV files contain text, category names and empty fields. We convert them in four steps: we fill the empty fields of each numeric column with that column's median, standardise the numeric columns, one-hot encode the categorical columns and join the blocks side by side. We compute the medians, means and standard deviations on the training rows only, so that the test rows cannot influence their own preprocessing.

THE THREE FILES

```
def read_rows(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))        # one dictionary per row, every value a string

train = read_rows("train.csv")        # the rows the models are trained on
test = read_rows("test.csv")          # held out, to measure a trained model
test2 = read_rows("test2.csv")        # held out, a second test set
len(train), len(test), len(test2)     # -> (4960, 1608, 3000)
train[0]
# -> {'age': '39', 'income': '32620.58', 'tenure_months': '24', 'score': '48.5',
#     'region': 'Harbour', 'plan': 'premium', 'channel': 'phone', 'churn': '1'}

ytr = np.array([int(r["churn"]) for r in train])     # (4960,) 1 for a customer who left
yte = np.array([int(r["churn"]) for r in test])      # (1608,)
y2 = np.array([int(r["churn"]) for r in test2])      # (3000,)
```

`csv.DictReader` reads the first line of the file as the column names. It returns each remaining line as a dictionary from column name to text, with an empty string where a value is missing. The three files have the same eight columns. The columns `age`, `income`, `tenure_months` and `score` are numeric. The columns `region`, `plan` and `channel` hold categories. The label `churn` is 1 for a customer who left and 0 for one who stayed. `test2.csv` holds 3000 further customers who are in neither of the other two files. All three files include the label. The accuracy of a model can therefore be computed on each of them.

THE FEATURE MATRIX

```
NUMERIC = ["age", "income", "tenure_months", "score"]
CATEGORICAL = ["region", "plan", "channel"]
CATEGORIES = {"region": ["North", "Harbour", "Midland", "Eastfield", "Southgate", "Upland"],
              "plan": ["basic", "standard", "premium", "legacy"],
              "channel": ["shop", "phone", "online"]}

def column(rows, name):
    return np.array([float(r[name]) if r[name] != "" else np.nan for r in rows])

medians = {c: float(np.nanmedian(column(train, c))) for c in NUMERIC}
medians     # -> {'age': 43.0, 'income': 27555.325, 'tenure_months': 23.0, 'score': 57.8}

def numeric_block(rows):
    cols = []
    for c in NUMERIC:
        v = column(rows, c)
        cols.append(np.where(np.isnan(v), medians[c], v))        # 1. gaps filled
    return np.stack(cols, axis=1)                                # (n rows, 4)

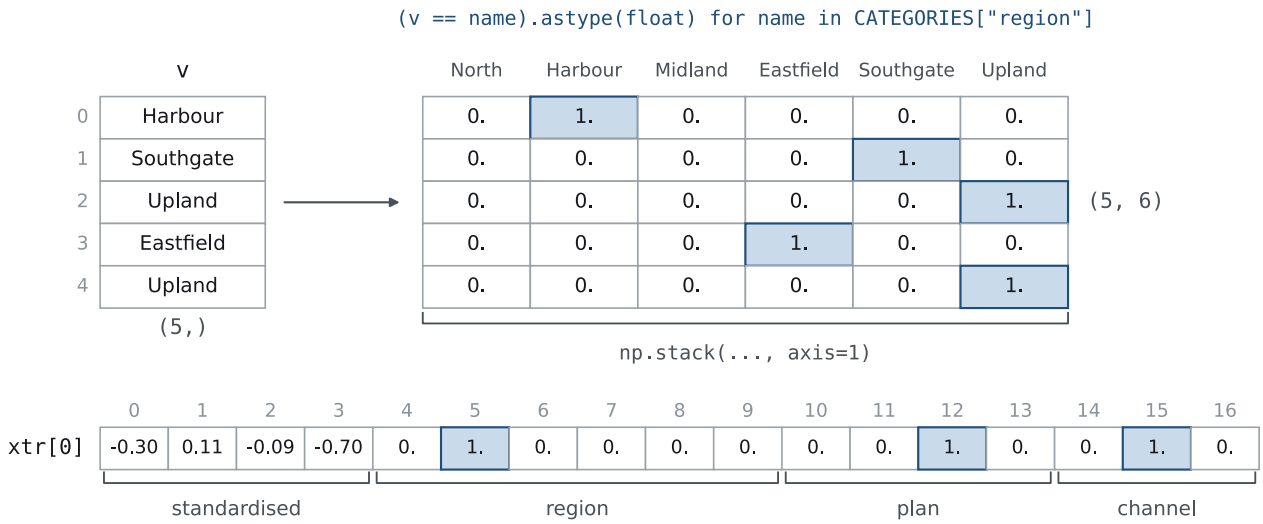
num_train = numeric_block(train)            # (4960, 4)
mean = num_train.mean(axis=0)               # -> [   42.93 31040.41    25.52    57.55]
std = num_train.std(axis=0)                 # -> [   13.11 14632.47    17.87    12.85]
```

The medians fill the gaps. The means and standard deviations standardise the filled columns. `features` applies all four steps to any list of rows, always with these statistics of the training rows:

```
def features(rows):
    blocks = [(numeric_block(rows) - mean) / std]                # 2. standardised
    for cat in CATEGORICAL:                                      # 3. one-hot
        v = np.array([r[cat] for r in rows])
        blocks.append(np.stack([(v == name).astype(float) for name in CATEGORIES[cat]],
                               axis=1))
    return np.concatenate(blocks, axis=1)                        # 4. joined

xtr = features(train)       # -> (4960, 17)   4 numeric + 6 + 4 + 3 one-hot
xte = features(test)        # -> (1608, 17)   the same 17 columns, in the same order
xtr[:, 0].mean()            # -> 0.0    age, on the training rows
xtr[:, 0].std()             # -> 1.0
```

`np.stack(cols, axis=1)` puts the four filled columns side by side as the columns of one `(n, 4)` array. A one-hot block is built the same way, from one comparison per category. `v == "North"` is a boolean array that is True on the rows of customers from North. `astype(float)` turns True into 1.0 and False into 0.0. Stacked along `axis=1`, the six columns of `region` give an `(n, 6)` block. Every row of the block holds a single 1, in the column of its region. `np.concatenate` then joins the standardised block and the three one-hot blocks along the columns, into 4 + 6 + 4 + 3 = 17 columns. `CATEGORIES` fixes the order of the one-hot columns. Column 7 therefore means Eastfield in every split, including a split in which no customer is from Eastfield.



Rows 0 to 4 of the region column and the six columns (v == name).astype(float), one per name in CATEGORIES["region"]. np.stack(..., axis=1) joins them into a (5, 6) block. Below is row 0 of xtr with its column indices: the four standardised columns, then the region, plan and channel blocks.

FITTING ON THE TRAINING SPLIT

The medians, the means and the standard deviations are computed on `train` and then applied unchanged to `test`. Recomputed on the test split, they would let the held-out rows influence their own preprocessing. The measured accuracy would then no longer be the accuracy of the model on rows it has never seen. `features` takes the rows as its argument and reads `medians`, `mean` and `std` from the training fit. The statistics therefore cannot come from the rows being transformed.

INDICATOR COLUMNS

```
for c in ["age", "income", "score"]:           # the numeric columns with gaps
    gap = np.isnan(column(train, c))
    print(f"{c:<7}{gap.sum():4} gaps   churn rate {ytr[gap].mean():.3f} with a gap, "
          f"{ytr[~gap].mean():.3f} without")
# age     303 gaps   churn rate 0.502 with a gap, 0.500 without
# income  404 gaps   churn rate 0.488 with a gap, 0.501 without
# score   213 gaps   churn rate 0.498 with a gap, 0.500 without
```

Filling a gap also erases the fact that the value was missing. An extra 0/1 indicator column per filled column would keep it. In these training rows the churn rate hardly depends on whether a value was missing, as the output above shows. The three indicator columns would add almost no information about the label, so we leave them out.

MIN-MAX SCALING

All the models in this handout use standardised features. Min-max scaling is the other common way to bring columns onto one scale, and the block below applies it to the training rows to show why we do not use it here.

```
lo, hi = num_train.min(axis=0), num_train.max(axis=0)    # (4,) each, from the training rows
minmax = (num_train - lo) / (hi - lo)                    # every training column onto [0, 1]
np.median(minmax[:, 1])                 # -> 0.157   the median income
np.quantile(minmax[:, 1], 0.95)         # -> 0.399
```

Min-max scaling maps each column's training minimum to 0 and its maximum to 1, which means the two most extreme values of a column decide its scale. For a column with a long tail, such as income, this squeezes most values into a small part of the range: the training maximum is 142611, so the median income becomes 0.157 and 95 per cent of the scaled incomes are below 0.399. Standardisation uses the mean and the standard deviation instead, which depend on every value in the column, so one extreme customer shifts them far less than it shifts the maximum. Whichever scaling we use, we compute its two numbers for each column on the training rows only, which is why rows from other splits can land outside [0, 1] after min-max scaling.

02

Splits and class balance

We fit the parameters on the training rows, but settings such as the number of epochs need rows of their own, so we hold back a random fifth of the training file as a validation set. We use the test rows only once, to measure the finished model: if we also used them to pick settings, we would pick whatever happens to work best on those particular rows, and the test accuracy would overstate how well the model does on new customers. All three files hold as many customers who left as customers who stayed, which means a model that always predicts the same class scores 0.5 on any of them.

A VALIDATION SPLIT

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(train))          # 0 to 4959 in a random order
val_rows = [train[i] for i in idx[:992]]   # the first fifth: validation
fit_rows = [train[i] for i in idx[992:]]   # the rest: fitting
len(fit_rows), len(val_rows)               # -> (3968, 992)
```

`np.random.default_rng(42)` creates a random number generator seeded with 42. Its `permutation(n)` returns the integers 0 to n - 1 in a random order. Slices of one permutation do not overlap. Each slice holds an exact number of rows, chosen at random. Splitting a file in its own order takes whatever order it was written in, by date, by customer number or by label. The two parts then differ in a systematic way.

CLASS BALANCE

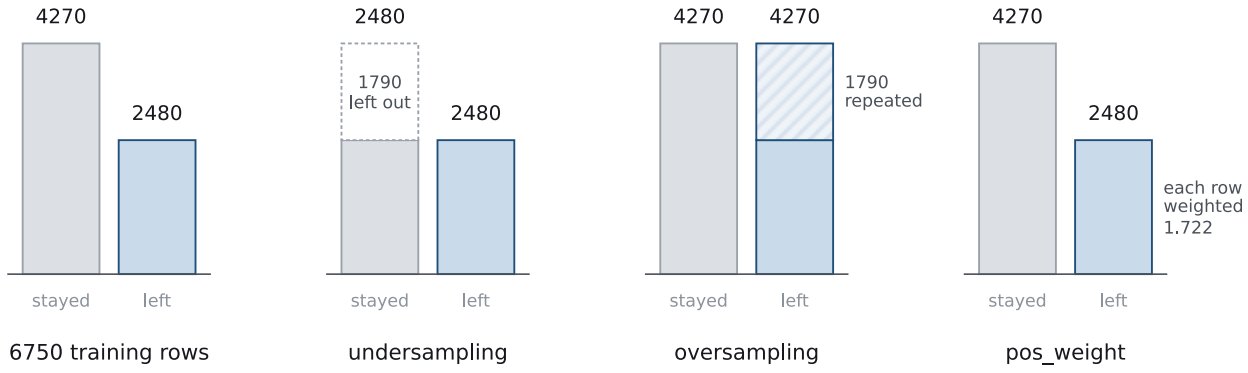
```
np.bincount(ytr)            # -> [2480 2480]   training rows: stayed, left
np.bincount(yte)            # -> [804 804]     test rows
np.bincount(y2)             # -> [1500 1500]   test2 rows
```

`np.bincount` counts the rows of each class, the customers who stayed first. Every file holds as many customers who left as customers who stayed. `train.csv` and `test.csv` were drawn from a register of 9000 customers. The register is not balanced. In it, 3284 customers left, 36 per cent. The register was first split into 6750 rows for training and 2250 for testing. The 6750 training rows held 2480 customers who left and 4270 who stayed. `train.csv` keeps all 2480 customers who left. It adds 2480 of the 4270 who stayed, drawn at random without repeats. This is **undersampling**. `test.csv` was made in the same way from the 2250 test rows. `test2.csv` holds 1500 customers of each class, drawn from a separate register of 9000 customers of the same population.

```
def undersample(y, rng):
    n = np.bincount(y).min()                    # the size of the smaller class
    keep = [rng.choice(np.flatnonzero(y == c), size=n, replace=False) for c in (0, 1)]
    return np.sort(np.concatenate(keep))        # row indices, in file order

rng = np.random.default_rng(0)
labels = rng.permutation(np.repeat([0, 1], [4270, 2480]))    # 4270 zeros, 2480 ones, mixed
np.bincount(labels)                           # -> [4270 2480]
rows = undersample(labels, rng)
np.bincount(labels[rows])                     # -> [2480 2480]
```

`np.repeat([0, 1], [4270, 2480])` writes 4270 zeros followed by 2480 ones. The permutation mixes them. The result is a label with the class counts of the 6750 training rows. `np.flatnonzero(y == c)` returns the indices of the rows of class `c`. `rng.choice(a, size=n, replace=False)` draws `n` different elements of `a`. From the larger class, `n` rows are drawn. The other rows of that class are left out. From the smaller class, all `n` rows are drawn, in a random order. `np.sort` puts the kept rows back in the order of the file.



The 6750 training rows of the register: 4270 stayed and 2480 left. The other three groups are the three ways of balancing the classes. Undersampling leaves out 1790 rows of the larger class. Oversampling repeats 1790 rows of the smaller class. pos_weight gives each row of the smaller class a weight of 4270 / 2480 in the loss.

Three methods are commonly used to balance the classes:

- **Undersampling.** Keep every row of the smaller class and a random sample of the same size from the larger one. Rows of the larger class are discarded.
- **Oversampling.** Draw rows of the smaller class with replacement until the classes are the same size. Rows of the smaller class are repeated.
- **Loss weighting.** Keep every row. Give the loss terms of the smaller class more weight.

```
weighted = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(4270 / 2480))   # 1.722
```

`pos_weight` multiplies the loss term of every row whose label is 1. The value 4270 / 2480 is the number of customers who stayed divided by the number who left. With it, the 2480 customers who left count as much in the loss as the 4270 who stayed. With balanced files, as here, `pos_weight` stays at its default of 1. The class balance of the test rows also sets the baseline. Always answering "stayed" scores 0.500 on these test rows. In the full register, 5716 of the 9000 customers stayed. The same answer would score 0.635 there.

03

Logistic regression and an MLP

Logistic regression computes a single weighted sum of the 17 features of a row, while a multilayer perceptron (MLP) first passes the features through hidden layers, each a weighted sum followed by ReLU, and then ends in the same kind of weighted sum. Since both return one number per row, we can train them with the same loss and the same loop and compare them directly, with logistic regression as the baseline for the MLP.

```
class LogReg(nn.Module):
    def __init__(self, n_in):
        super().__init__()
        self.linear = nn.Linear(n_in, 1)        # one weighted sum, no hidden layer

    def forward(self, x):                       # (n, n_in) -> (n,)
        return self.linear(x).squeeze(-1)

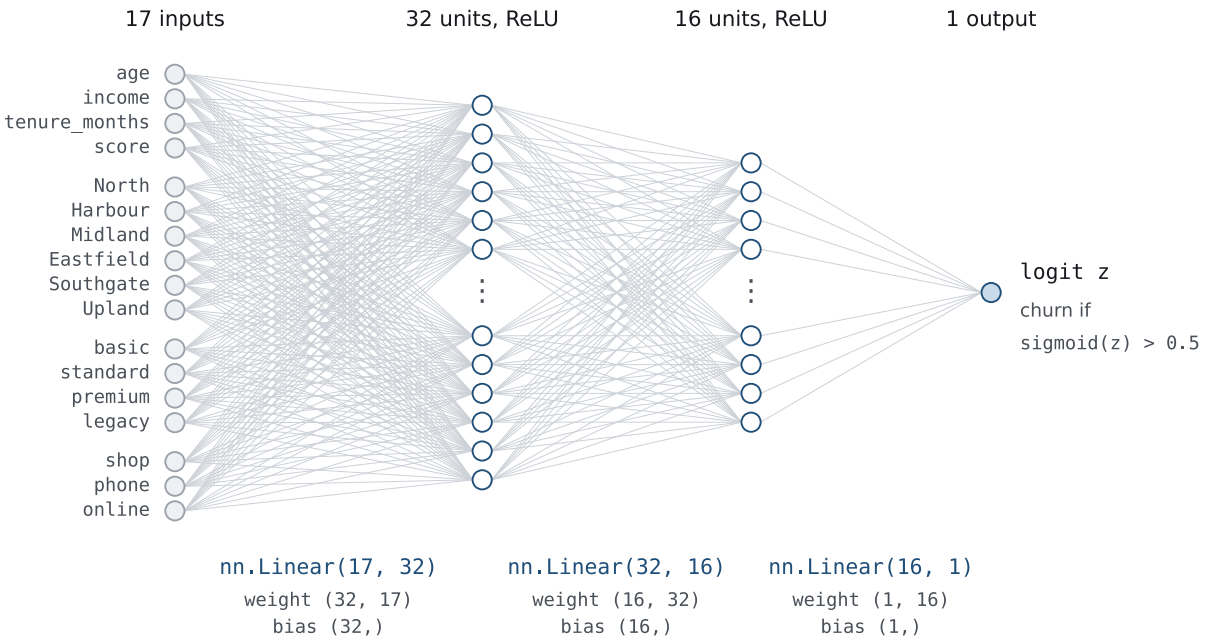
class Net(nn.Module):
    def __init__(self, n_in):
        super().__init__()
        self.stack = nn.Sequential(
            nn.Linear(n_in, 32), nn.ReLU(),
            nn.Linear(32, 16), nn.ReLU(),
            nn.Linear(16, 1))

    def forward(self, x):                       # (n, n_in) -> (n,)
        return self.stack(x).squeeze(-1)

torch.manual_seed(42)
net = Net(17)
sum(p.numel() for p in LogReg(17).parameters())     # -> 18
sum(p.numel() for p in net.parameters())            # -> 1121

[(name, tuple(p.shape)) for name, p in net.named_parameters()]
# -> [('stack.0.weight', (32, 17)), ('stack.0.bias', (32,)),
#     ('stack.2.weight', (16, 32)), ('stack.2.bias', (16,)),
#     ('stack.4.weight', (1, 16)), ('stack.4.bias', (1,))]
```

Logistic regression is one weighted sum of the 17 features plus a bias, 18 parameters. The sigmoid of the sum is read as the probability that the label is 1. `Net` puts two hidden layers, of 32 and 16 units with ReLU, in front of the same kind of last layer. Its parameters are 17 by 32 plus 32, then 32 by 16 plus 16, then 16 by 1 plus 1, 1121 in all. The last layer of both models gives an output of shape `(n, 1)`. `squeeze(-1)` removes the last axis to give the output the shape of the labels, `(n,)`.



Net(17): the 17 columns of the feature matrix, hidden layers of 32 and 16 units with ReLU, and one output, the logit. Each line is one weight. Some units of the hidden layers are left out of the drawing. Each nn.Linear(a, b) holds a weight of shape (b, a) and a bias of shape (b,). The six tensors hold 544 + 32 + 512 + 16 + 16 + 1 = 1121 parameters.

TEST ACCURACY OF THE TWO MODELS

Both models are trained for 40 epochs with Adam at a learning rate of 0.001, on minibatches of 128 rows. Logistic regression then scores 0.664 on the test rows and 0.680 on the rows of `test2.csv`. The MLP scores 0.659 and 0.678. Over the seeds 0 to 4, the MLP scores between 0.652 and 0.660 on the test rows. Logistic regression scores between 0.664 and 0.667. On this table the hidden layers add nothing measurable. The 18-parameter model is therefore the baseline that a deeper network is compared against.

04

Logits and BCEWithLogitsLoss

Both models end in a linear layer with nothing after it, so their output is a **logit**: a real number that a sigmoid would turn into a probability. We leave the sigmoid out of the model because `nn.BCEWithLogitsLoss` applies it itself, in a numerically stable form, before comparing the result with the label, and the loss it returns is large when the prediction is wrong.

The final `nn.Linear(16, 1)` of `Net`, like the single layer of `LogReg`, produces one logit per row. It can be any real number. The sigmoid, `sigmoid(z) = 1 / (1 + exp(-z))`, maps it to a probability between 0 and 1. The logit and the probability describe the same prediction.

```
torch.sigmoid(torch.tensor([0.0, 2.0, -2.0]))   # -> tensor([0.5000, 0.8808, 0.1192])
```

A logit of 0 is a probability of a half. Positive logits give probabilities above a half. Negative logits give probabilities below a half.

BINARY CROSS-ENTROPY

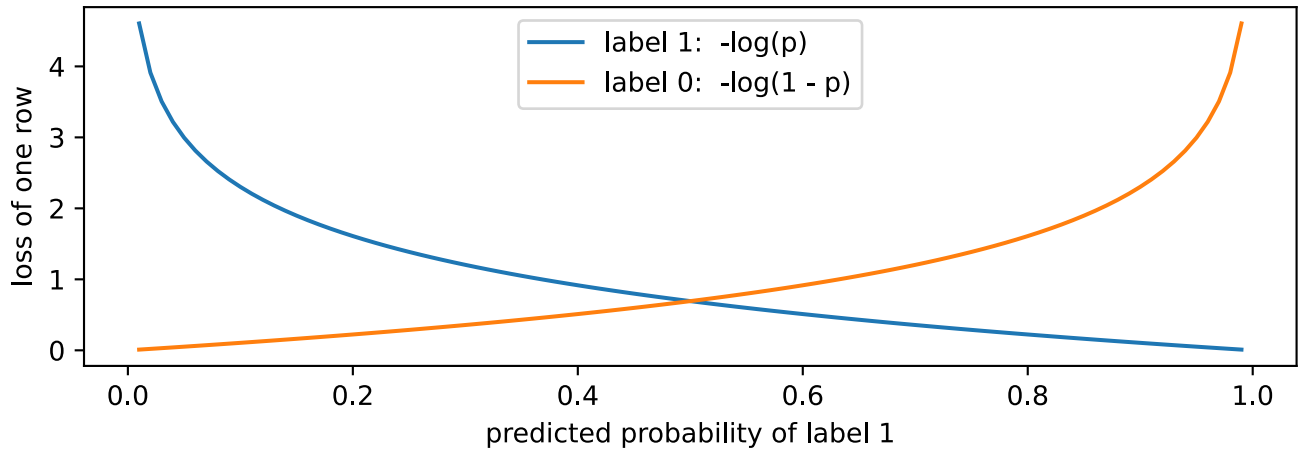
For one row with label `y`, 0 or 1, and predicted probability `p` of label 1, the **binary cross-entropy** is

$$\ell(p, y) = -\big[\, y \log p + (1 - y) \log(1 - p) \,\big], \qquad p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$L = \frac{1}{n} \sum_{i=1}^{n} \ell(p_i, y_i)$$

For a row with label 1, only the first term remains, `-log(p)`, and for a row with label 0 only the second, `-log(1 - p)`. Each term is 0 when the probability given to the true label is 1, it is `-log(0.5) = ln 2 = 0.693` when that probability is a half, and it grows without limit as the probability goes to 0. A row predicted at 0.9 for its true label adds 0.105 to the loss, while a row predicted at 0.1 adds 2.303. The loss of a batch is the mean over its `n` rows.

```
p = np.linspace(0.01, 0.99, 99)                # probabilities of label 1
plt.figure(figsize=(7, 2.6))
plt.plot(p, -np.log(p), label="label 1:  -log(p)")
plt.plot(p, -np.log(1 - p), label="label 0:  -log(1 - p)")
plt.xlabel("predicted probability of label 1")
plt.ylabel("loss of one row")
plt.legend()
plt.tight_layout()
plt.show()
```



BCEWITHLOGITSLOSS

```
loss_fn = nn.BCEWithLogitsLoss()               # takes logits
x = torch.tensor(xtr, dtype=torch.float32)     # (4960, 17)
y = torch.tensor(ytr, dtype=torch.float32)     # (4960,) of 0.0 / 1.0, a float
loss_fn(net(x), y).item()                      # -> 0.6930   (n,) and (n,) -> one number
```

`nn.BCELoss()(torch.sigmoid(z), y)` computes the loss from probabilities. `nn.BCEWithLogitsLoss()(z, y)` takes the logits. It combines the sigmoid and the loss in one class and never computes `p` itself. Its label argument is a float of 0.0 or 1.0, not an integer.

WHY THE LOSS TAKES LOGITS

We pass logits to the loss so that it never has to compute `p`. If we computed `p = sigmoid(z)` first and then took its logarithm, a row the model gets badly wrong would break the computation: at a logit of -100 the float32 sigmoid is exactly 0, and the logarithm of 0 is minus infinity. Instead we write both logarithms of the loss with the logit alone. We start from the sigmoid and from `1 - sigmoid(z)`, which we bring onto the same denominator:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$1-\sigma(z) = \frac{1+e^{-z}}{1+e^{-z}} - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$$

The logarithm of a fraction is the logarithm of its numerator minus the logarithm of its denominator. With `log 1 = 0` and `log exp(-z) = -z`, the two logarithms in the loss become

$$\log \sigma(z) = \log 1 - \log\left(1 + e^{-z}\right) = -\log\left(1 + e^{-z}\right)$$
$$\log\left(1 - \sigma(z)\right) = \log e^{-z} - \log\left(1 + e^{-z}\right) = -z - \log\left(1 + e^{-z}\right)$$

We put both into the binary cross-entropy above. The minus sign in front of its bracket cancels their minus signs, and `log(1 + exp(-z))` then appears once with the factor `y` and once with `1 - y`. The two factors add up to 1, so what is left depends only on `z` and `y`:

$$\ell(z, y) = y \log\left(1 + e^{-z}\right) + (1 - y)\left[z + \log\left(1 + e^{-z}\right)\right]$$
$$= (1 - y)\, z + \log\left(1 + e^{-z}\right)$$

Written this way, the loss still breaks for a large negative logit, because `exp(-z)` overflows: `exp(89)` is already infinite in float32, so the loss comes out as `inf`. The standard fix is the **log-sum-exp trick**, which takes the largest exponent `m` out of a sum of exponentials as a factor `exp(m)`. Because `m` is the largest exponent, `a - m` and `b - m` are both at most 0, so no exponential left inside the sum is larger than 1:

$$e^{a} + e^{b} = e^{m}\left(e^{a-m} + e^{b-m}\right)$$
$$\log\left(e^{a} + e^{b}\right) = m + \log\left(e^{a-m} + e^{b-m}\right), \qquad m = \max(a, b)$$

In the loss, `1 + exp(-z) = exp(0) + exp(-z)`, which gives `a = 0` and `b = -z`. The larger of the two, `m = max(0, -z)`, depends on the sign of `z`, so we take the two cases one at a time:

$$z \geq 0,\ m = 0: \quad \log\left(1 + e^{-z}\right) = \log\left(1 + e^{-|z|}\right)$$
$$z < 0,\ m = -z: \quad \log\left(1 + e^{-z}\right) = -z + \log\left(e^{z} + 1\right) = -z + \log\left(1 + e^{-|z|}\right)$$

In both cases the term in front of the logarithm is `max(0, -z)` and the only exponential left is `exp(-|z|)`, so the loss becomes

$$\ell(z, y) = (1 - y)\, z + \max(0, -z) + \log\left(1 + e^{-|z|}\right)$$

The only exponential left, `exp(-|z|)`, lies between 0 and 1 for every logit, so nothing can overflow any more. In PyTorch, `torch.clamp(-z, min=0)` computes `max(0, -z)` element by element, and `z.abs()` computes `|z|`:

```
for v in [-1000.0, -20.0, 0.0, 20.0]:       # rows with label 1, so (1 - y) * z = 0
    z = torch.tensor([v])
    naive = torch.log(1 + torch.exp(-z))
    stable = torch.clamp(-z, min=0) + torch.log(1 + torch.exp(-z.abs()))
    combined = nn.BCEWithLogitsLoss()(z, torch.ones(1))
    print(f"logit {v:7.1f}   naive {naive.item():9.4f}   stable {stable.item():9.4f}   "
          f"BCEWithLogitsLoss {combined.item():9.4f}")
# logit -1000.0   naive       inf   stable 1000.0000   BCEWithLogitsLoss 1000.0000
# logit   -20.0   naive   20.0000   stable   20.0000   BCEWithLogitsLoss   20.0000
# logit     0.0   naive    0.6931   stable    0.6931   BCEWithLogitsLoss    0.6931
# logit    20.0   naive    0.0000   stable    0.0000   BCEWithLogitsLoss    0.0000
```

The documentation of `nn.BCEWithLogitsLoss` names the log-sum-exp trick as the reason it is more stable than a sigmoid followed by `nn.BCELoss`, and the output above shows that its values agree with the stable form. Computing the loss from the logit also changes the gradient:

```
for v in [-20.0, -40.0, -100.0]:                # one row with label 1, ever more wrong
    z1 = torch.tensor([v], requires_grad=True)
    z2 = torch.tensor([v], requires_grad=True)
    loss1 = nn.BCEWithLogitsLoss()(z1, torch.ones(1))          # combined
    loss2 = nn.BCELoss()(torch.sigmoid(z2), torch.ones(1))     # sigmoid, then the loss
    loss1.backward()
    loss2.backward()
    print(f"logit {v:6.1f}   losses {loss1.item():5.1f} {loss2.item():5.1f}   "
          f"gradients {z1.grad.item():4.1f} {z2.grad.item():9.6f}")
# logit  -20.0   losses  20.0  20.0   gradients -1.0 -1.000000
# logit  -40.0   losses  40.0  40.0   gradients -1.0 -0.000004
# logit -100.0   losses 100.0 100.0   gradients -1.0 -0.000000
```

The two give the same loss and different gradients. For a row with label `y`, the gradient of the loss with respect to the logit is `sigmoid(z) - y`. For label 1 it tends to -1 as the logit goes to minus infinity. The combined version computes the gradient in this form. The separate version differentiates through the probability `sigmoid(z)`. At a logit of -40, `sigmoid(z)` is 4.2e-18. The separate version then returns a gradient of -0.000004. At -100, the float32 sigmoid returns exactly 0. The gradient is then 0 as well. The loss stays at 100 because `BCELoss` clamps its logarithms at -100. A row the model gets badly wrong then produces almost no update, or none.

THE LOSS BEFORE TRAINING

An untrained network outputs logits close to 0. The sigmoid turns them into probabilities close to a half. Every row then adds about `-log(0.5) = ln 2 = 0.6931` to the loss, whatever its label. The untrained `Net(17)` above gives 0.6930 on the 4960 training rows. On the first minibatch of the training loop it gives 0.6912.

The same reasoning holds for any number of classes. With `K` classes a network has one output per class, and the cross-entropy of a row is `-log` of the probability it gives the true class. An untrained network gives every class about `1/K`, so its loss starts near `-log(1/K) = ln K`: 1.0986 for three classes, 1.3863 for four and 2.3026 for ten. We use this as a check before training. If the first loss is far from `ln K`, the logits of the untrained network are not close to 0, and the usual cause is the input: an income in tens of thousands, multiplied by even small random weights, gives large logits on unscaled features, so many rows get a probability near 0 for their true class and the first loss comes out far above `ln K`.

The value `ln K` is also the loss that training has to get below, but only when the classes are equally large. A model that ignores the features can still learn how often each class occurs and predict those shares for every row. With equally large classes, as in our three files, the shares are all `1/K` and its loss stays at `ln K`. With unequal classes the shares help: in the full register 36.5 per cent of the labels are 1, and a model that predicts 0.365 for every row adds `-ln 0.365` to the loss for each row with label 1 and `-ln 0.635` for each row with label 0. Its mean loss is `-(0.365 ln 0.365 + 0.635 ln 0.635) = 0.656`, below `ln 2 = 0.693`, so on such data a network whose loss stays between 0.656 and 0.693 has learned the class shares and nothing from the features.

05

The training loop

In each training step we take one **minibatch**, a random sample of the training rows, compute the loss on it and change every parameter once using the gradient of that loss. The step itself is four statements: `opt.zero_grad()`, the forward pass, `loss.backward()` and `opt.step()`. The inner loop below runs one step per minibatch until it has used every training row once, which is one **epoch**, and the outer loop repeats this for 40 epochs.

```
torch.set_num_threads(4)                          # 4 threads, as for the values below
torch.manual_seed(42)                             # the same weights and batches on every run
net = Net(xtr.shape[1])
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
loss_fn = nn.BCEWithLogitsLoss()

x = torch.tensor(xtr, dtype=torch.float32)        # (4960, 17)
y = torch.tensor(ytr, dtype=torch.float32)        # (4960,)

losses = []
for epoch in range(40):
    perm = torch.randperm(len(x))                 # reshuffle every epoch
    for i in range(0, len(x), 128):
        b = perm[i:i + 128]                       # the row indices of one minibatch
        opt.zero_grad()                           # 1. clear last step's gradients
        loss = loss_fn(net(x[b]), y[b])           # 2. forward pass and loss
        loss.backward()                           # 3. gradients of the loss
        opt.step()                                # 4. update the parameters from .grad
        losses.append(loss.item())                # the value, as a Python float
```

STATEMENT	EFFECT	EFFECT IF OMITTED
`opt.zero_grad()`	Sets every `.grad` to `None`	Each step follows the sum of all earlier gradients. The loss soon stops falling.
`loss = loss_fn(net(x), y)`	Predicts, then scores the prediction	`backward()` has no new loss to differentiate.
`loss.backward()`	Fills every `.grad`	`.grad` stays `None`. `step()` then changes nothing.
`opt.step()`	Updates parameters from `.grad`	Gradients are computed and discarded. The parameters never change.

		stack.4.weight[0, :3]			its .grad			loss
	the state before	-0.1069	0.1506	0.0491	0.00066	0.0	0.00501	0.6912
next minibatch	opt.zero_grad()	-0.1069	0.1506	0.0491	None			0.6912
	loss = loss_fn(net(x[b]), y[b])	-0.1069	0.1506	0.0491	None			0.6947
	loss.backward()	-0.1069	0.1506	0.0491	-0.00029	0.0	0.00023	0.6947
	opt.step()	-0.1072	0.1506	0.0484	-0.00029	0.0	0.00023	0.6947

The second step of the loop above, statement by statement, for three weights of the last layer, stack.4.weight[0, :3]. Shaded cells are the ones the statement changes. zero_grad() empties .grad. The forward pass computes the loss of the batch. backward() fills .grad. step() changes the weights. Adam's step also uses the gradient of the first step. The first weight therefore falls although its new gradient is negative. The middle weight reads a hidden unit that is 0 on every row of the batch. Its gradient is 0. step() leaves it unchanged.

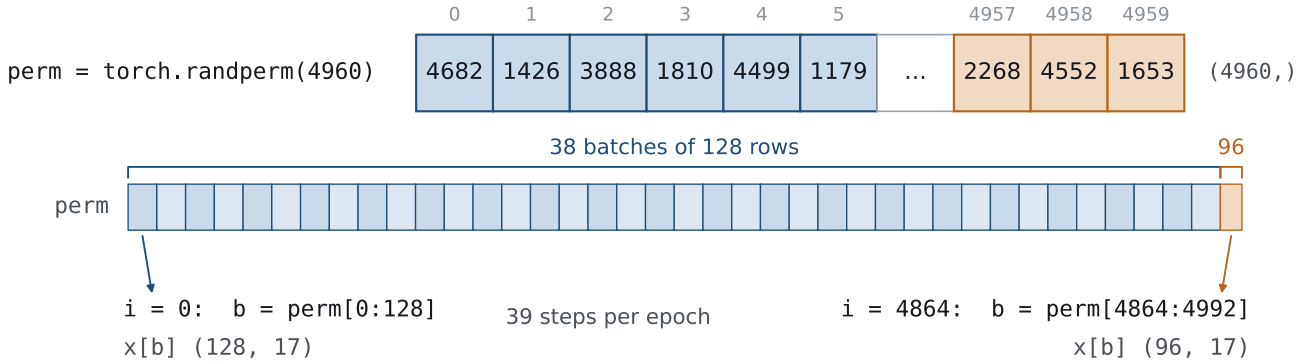
NET.PARAMETERS()

`net.parameters()` yields every tensor the module registered. The optimizer holds references to these tensors. `step()` changes them in place. The model then computes with the changed values. Only registered tensors are handed to the optimizer. A layer that was never registered, such as one kept in a plain Python list, is never trained. No error is raised.

MINIBATCHES

One step per row makes 4960 steps per epoch, each computed from a single row. One step per epoch over all 4960 rows makes 40 steps in 40 epochs. A batch of 128 rows, as here, makes 39 steps per epoch. Each step

uses an estimate of the gradient from a sample of rows. Batch sizes are usually powers of two, such as 32, 64, 128 or 256. The largest usable size is set by the memory of the device. The forward pass keeps the intermediate results of every row of the batch until `backward()` has used them. `torch.randperm` draws a new order in every epoch. The batches therefore differ from one epoch to the next.

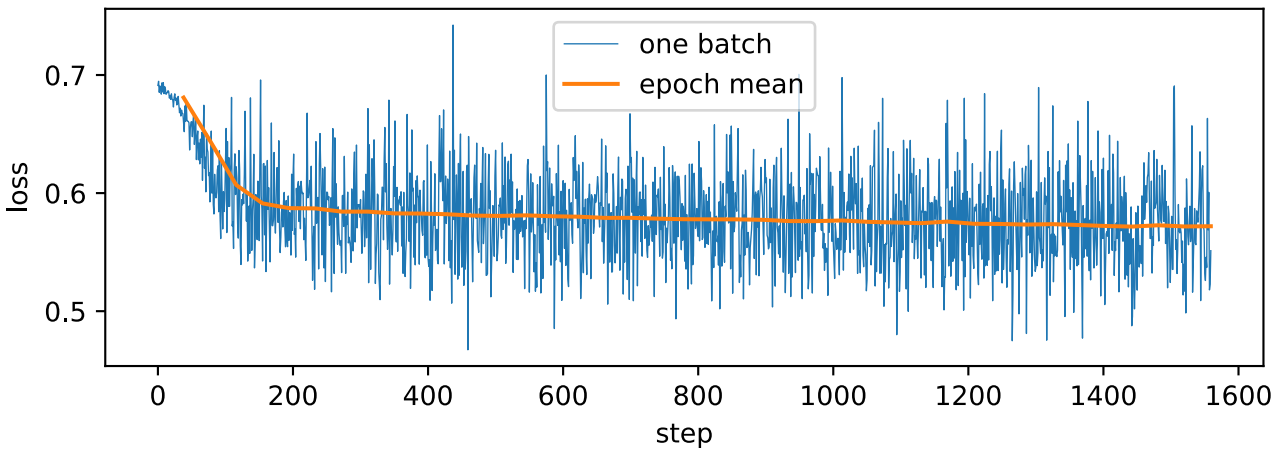


The first epoch's torch.randperm(4960) cut into batches by perm[i:i + 128]: 38 batches of 128 row indices and a last one of 96, cut short by the end of perm. Each batch selects its rows of x for one step.

RECORDING AND PLOTTING THE LOSS

`loss.item()` returns the value of a one-element tensor as a Python float. Appending the tensor itself would also keep its part of the graph in memory, through `grad_fn`. The float holds only the number. Forty epochs of 39 batches give 1560 values. The last batch of each epoch holds the 96 rows left over after 38 batches of 128.

```
plt.figure(figsize=(7, 2.6))
plt.plot(losses, linewidth=0.5, label="one batch")
means = np.array(losses).reshape(40, 39).mean(axis=1)     # (1560,) -> (40, 39) -> (40,)
plt.plot(np.arange(38, 1560, 39), means, label="epoch mean")
plt.xlabel("step")
plt.ylabel("loss")
plt.legend()
plt.tight_layout()
plt.show()
```



`plt.plot(v)` draws a line through the values of `v` against 0, 1, 2 and so on. `plt.plot(x, v)` takes the x positions explicitly. Here the epoch means are drawn at the last step of each epoch. `reshape(40, 39)` puts one epoch per row. The mean along axis 1 is then the mean batch loss of each epoch.

READING THE LOSS

The loss varies from batch to batch, because each batch is a different sample of rows. The loss is 0.6912 on the first batch and 0.5512 on the last. The epoch means fall from 0.681 to 0.572, quickly in the first three epochs and slowly after that. A loss that rises steadily usually means the learning rate is too high. A loss that does not move usually means a missing `backward()`, a missing `step()` or unscaled features.

`torch.set_num_threads(4)`, the first line of the training loop above, fixes how many threads PyTorch uses on the CPU. The values from the training loop on were computed with 4 threads. With another number of threads, the floating-point sums are rounded differently. A few of the values then differ. PyTorch also does not guarantee identical results across platforms.

06

Dataset and DataLoader

So far we have shuffled the row indices and sliced out each minibatch by hand. PyTorch splits that work between two classes: a `Dataset` returns one row for a given index, and a `DataLoader` shuffles the indices and stacks the returned rows into batches. To write a `Dataset` we only need `__len__` and `__getitem__`, the two methods that make any object behave like a sequence.

```
from torch.utils.data import Dataset, DataLoader

class Rows(Dataset):
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __len__(self):                      # how many rows there are
        return len(self.x)

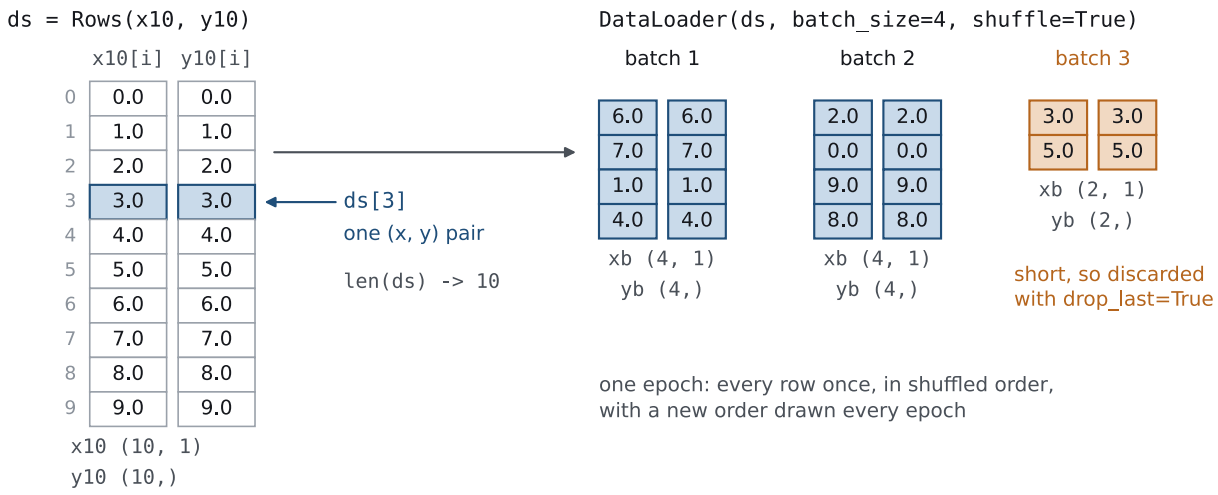
    def __getitem__(self, i):               # ONE row, as a tuple
        return self.x[i], self.y[i]
```

The class returns one row and does no batching or shuffling. The `DataLoader` adds both, as ten rows of one feature show:

```
x10 = torch.arange(10.0).reshape(10, 1)     # 10 rows of 1 feature, x10[i] = i
y10 = torch.arange(10.0)                    # 10 targets, y10[i] = i
ds = Rows(x10, y10)
len(ds)                 # -> 10
ds[3]                   # -> (tensor([3.]), tensor(3.))   one row, through __getitem__

dl = DataLoader(ds, batch_size=4, shuffle=True)
len(dl)                 # -> 3     batches of 4, 4 and 2
[tuple(xb.shape) for xb, yb in dl]      # -> [(4, 1), (4, 1), (2, 1)]
len(DataLoader(ds, batch_size=4, drop_last=True))     # -> 2   the short batch discarded
```

The loader stacks the tuples that `__getitem__` returns into batched tensors. With `shuffle=True`, it reshuffles the rows every epoch. It yields batches until the rows run out. The last batch is short unless `drop_last=True`. Dropping it matters when a layer needs a fixed batch size, or when a short final batch would make a running average jump.



Left: ds holds 10 pairs, here with x10[i] = y10[i] = i. ds[i] returns one of them. Right: one shuffled epoch of DataLoader(ds, batch_size=4, shuffle=True). The loader stacks the pairs into batches of 4, 4 and 2 rows.

THE TRAINING LOOP WITH A DATALOADER

```
loader = DataLoader(Rows(x, y), batch_size=128, shuffle=True)
len(loader)             # -> 39   batches per epoch

torch.manual_seed(42)
net = Net(17)
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for epoch in range(40):
    for xb, yb in loader:                   # shuffling and batching, both handled
        opt.zero_grad()
        loss = loss_fn(net(xb), yb)
        loss.backward()
        opt.step()
round(loss.item(), 4)   # -> 0.5600   the last batch
```

The four statements are unchanged. `torch.randperm` and the index arithmetic are no longer in the loop. The loader draws its own random order. With the same seed, its batches therefore differ from those of the hand-written loop. The loss of its last batch is 0.5600, against 0.5512 in the hand-written loop. For tensors already in memory, `TensorDataset(x, y)` from the same module does the same without a new class. A class of one's own lets `__getitem__` read a file, decode an image or apply a transformation one row at a time. Data too large for memory needs such a class.

A GENERATOR OF MINIBATCHES

```
def minibatches(n, size):
    perm = torch.randperm(n)                # a new order on every call
    for i in range(0, n, size):
        yield perm[i:i + size]

torch.manual_seed(42)                       # the same start as the hand-written loop
net = Net(17)
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for epoch in range(40):
    for b in minibatches(len(x), 128):
        opt.zero_grad()
        loss = loss_fn(net(x[b]), y[b])
        loss.backward()
        opt.step()
round(loss.item(), 4)   # -> 0.5512   the last batch, as in the hand-written loop
```

The shuffling and slicing of the hand-written loop can also move into a generator function. Only the four statements are then left in the loop. Every call of `minibatches` draws a new order with `torch.randperm`. With the same seed, it therefore produces exactly the batches, the losses and the trained weights of the hand-written loop.

07

Evaluation

The accuracy of a model is the share of rows on which it predicts the label correctly. We measure it on rows the model was not trained on, because a model can fit its training rows without learning anything that carries over, and we compare it with the accuracy of rules that ignore the features, such as always predicting the same class.

```
def predict(matrix):
    with torch.no_grad():                                # no gradient recording
        logits = net(torch.tensor(matrix, dtype=torch.float32))
        return (torch.sigmoid(logits) > 0.5).long().numpy()

pred = predict(xte)                 # (1608,) of 0 / 1
(pred == yte).mean()                # -> 0.6586   accuracy on the test rows
```

`predict` runs under `torch.no_grad()`, because nothing is learned while it runs. `torch.sigmoid` turns the logits into probabilities. **A threshold of 0.5** turns the probabilities into answers. The threshold can be changed. A higher threshold predicts churn less often. It finds fewer of the churners and raises fewer false alarms.

ACCURACY AGAINST THE BASE RATE

Half of the test rows are customers who left. The table compares the accuracy of the models with simpler rules on these rows:

MODEL	ACCURACY
Always answering "stayed"	`0.500`
Answering at random	`0.500`
Logistic regression, test split	`0.664`
The network above, test split	`0.659`
The same network, the 3000 rows of `test2.csv`	`0.678`

On an *unbalanced* split where 80 per cent do not churn, the constant answer scores 0.80. A model at 0.78 is then below it. The constant answer always scores the share of the larger class.

A SECOND TEST SET

```
(predict(features(test2)) == y2).mean()     # -> 0.678
```

The same trained model scores 0.678 on the 3000 rows of `test2.csv` and 0.659 on the 1608 rows of `test.csv`. Both files were drawn from the same population. The difference is of the size that sampling alone produces. An accuracy near 0.66, measured on 1608 random rows, varies from one sample to the next with a standard deviation of about `(0.66 * 0.34 / 1608) ** 0.5 = 0.012`. On 3000 rows the standard deviation is about 0.009.

PREPROCESSING STATISTICS AND PREDICTED CLASS COUNTS

- **Preprocessing statistics from the training rows only.** The medians, means and standard deviations are computed on the training rows. A statistic computed over the whole table before the split also includes the

test rows.

- **Predicted class counts.** A model that predicts a single class for every row can still reach a high accuracy on an unbalanced split. `np.bincount(pred)` shows how many rows were given each class.

08

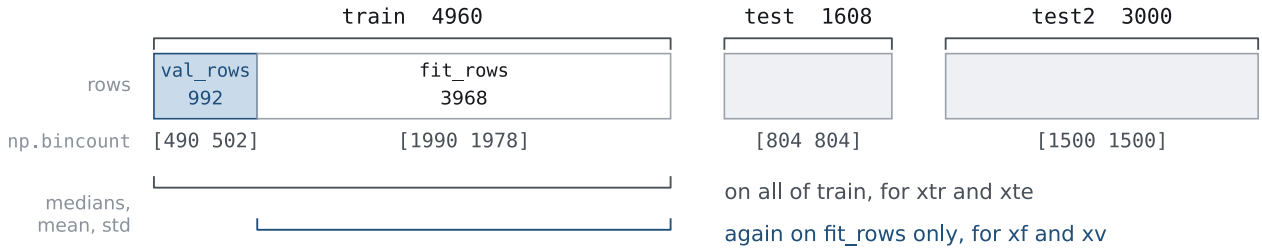
Validation and early stopping

The training loss tells us how well the network fits the rows it is trained on, and it keeps falling for as long as we train. To see whether that fit carries over to new rows, we also compute the loss on the validation rows after every epoch. Once the validation loss stops falling, further epochs only help on the training rows, so we use it to decide how long to train.

```
medians = {c: float(np.nanmedian(column(fit_rows, c))) for c in NUMERIC}
num_train = numeric_block(fit_rows)
mean, std = num_train.mean(axis=0), num_train.std(axis=0)

xf, xv = features(fit_rows), features(val_rows)          # (3968, 17), (992, 17)
yf = np.array([int(r["churn"]) for r in fit_rows])
yv = np.array([int(r["churn"]) for r in val_rows])
x, y = torch.tensor(xf, dtype=torch.float32), torch.tensor(yf, dtype=torch.float32)
xv_t, yv_t = torch.tensor(xv, dtype=torch.float32), torch.tensor(yv, dtype=torch.float32)
```

The medians, means and standard deviations are fitted again on `fit_rows` alone. Nothing about the validation rows then reaches the model. Both sets are converted to tensors once, before the loop.



The rows of the three files to scale, with np.bincount of the labels of each part as [stayed left]. The brackets mark the rows the medians, means and standard deviations are computed from.

TRAINING AND VALIDATION LOSS PER EPOCH

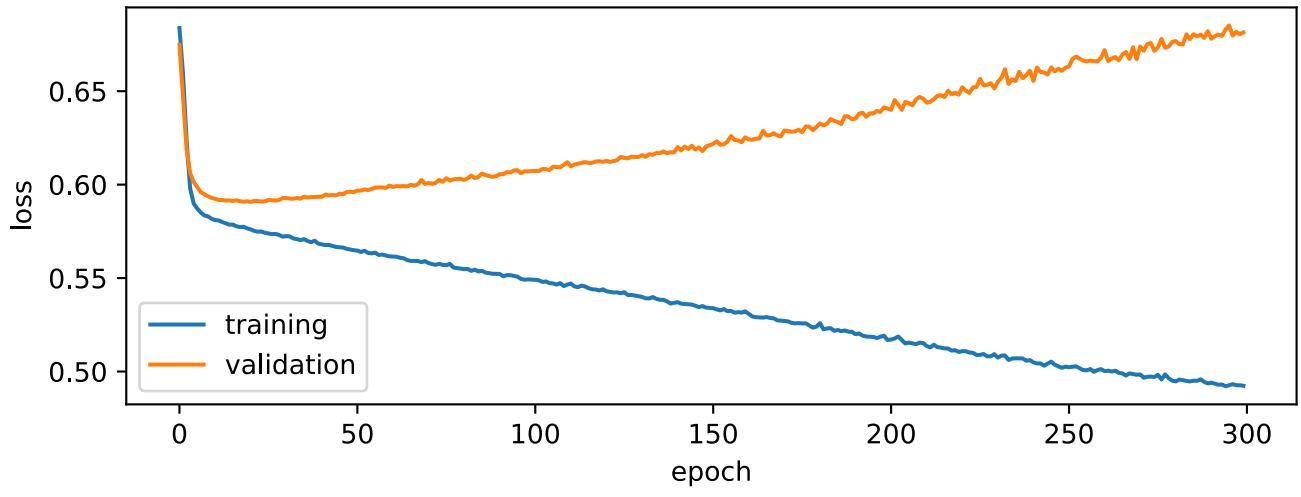
```
def train_epoch(net, opt, x, y):
    net.train()
    perm = torch.randperm(len(x))
    losses = []
    for i in range(0, len(x), 128):
        b = perm[i:i + 128]
        opt.zero_grad()
        loss = loss_fn(net(x[b]), y[b])
        loss.backward()
        opt.step()
        losses.append(loss.item())
    return np.mean(losses)                  # the mean loss of the epoch

def evaluate(net, x, y):
    net.eval()
    with torch.no_grad():
        return loss_fn(net(x), y).item()

torch.manual_seed(42)
net = Net(17)
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
history = []
for epoch in range(300):
    history.append((train_epoch(net, opt, x, y), evaluate(net, xv_t, yv_t)))
```

The loop uses two functions. `train_epoch` runs one epoch of the minibatch loop and returns the mean batch loss. `evaluate` computes the loss on the validation rows without recording a graph. `net.train()` and `net.eval()` set a flag on the module. Layers such as dropout and batch normalisation read the flag, because they behave differently during training. This network has neither kind of layer. The two calls change nothing here. They keep the loop correct for a network with such layers.

```
h = np.array(history)                       # (300, 2): training, validation
plt.figure(figsize=(7, 2.8))
plt.plot(h[:, 0], label="training")
plt.plot(h[:, 1], label="validation")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.legend()
plt.tight_layout()
plt.show()
```



OVERFITTING

The validation loss falls with the training loss for about ten epochs and then levels off. Its lowest value, 0.591, comes in epoch 21. From epoch 10 to epoch 40 it stays within 0.0028 of that value. After epoch 40 the two curves part. The training loss keeps falling, to 0.492 by epoch 300. The validation loss rises, to 0.607 at epoch 100 and 0.681 at epoch 300. The network is fitting the particular rows it is trained on rather than the relationship between the features and the label. This is called **overfitting**. Training for 40 epochs ends at the bottom of the validation curve.

EARLY STOPPING

```
torch.manual_seed(42)
net = Net(17)
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
best_loss, best_epoch, best_state = float("inf"), 0, None
for epoch in range(300):
    train_epoch(net, opt, x, y)
    val_loss = evaluate(net, xv_t, yv_t)
    if val_loss < best_loss:
        best_loss, best_epoch = val_loss, epoch
        best_state = copy.deepcopy(net.state_dict())
    elif epoch - best_epoch >= 20:          # 20 epochs without improvement
        break
net.load_state_dict(best_state)             # the weights of the best epoch

epoch, best_epoch, round(best_loss, 4)      # -> (40, 20, 0.5908)
(predict(features(test)) == yte).mean()     # -> 0.6598
```

Early stopping monitors the validation loss during training. It keeps the weights of the best epoch so far. It stops once the validation loss has not improved for a set number of epochs, the patience. With a patience of 20, this run stops after epoch 41 and restores the weights of epoch 21. Counted from 0, these are the epochs 40 and 20 that the code prints. `state_dict()` returns a dictionary of the parameter tensors themselves, not copies. Without `copy.deepcopy`, the saved state would change with every later update. It would end as the last weights rather than the best ones. With the restored weights, the network scores 0.660 on the test rows. The 40-epoch run on all 4960 training rows scores 0.659.

09

Saving, loading and the GPU

When we save a trained network, only its parameters go into the file, and the class that defines it stays in our code. To load the network, we create a new instance of the same class and copy the saved parameters into it, which is why the class definition has to be kept together with the file.

```
torch.save(net.state_dict(), "churn_net.pt")    # the parameters, by name

net2 = Net(17)                                  # a new instance with random weights
net2.load_state_dict(torch.load("churn_net.pt"))
net2.eval()
torch.equal(net(x), net2(x))                    # -> True   the same outputs, bit for bit
```

`state_dict()` maps each parameter name to its tensor, `stack.0.weight` to its `(32, 17)` matrix and so on. `torch.save` writes it to a file, under 8 kB for this network. `load_state_dict` copies the tensors into a new instance. The new instance then computes exactly what the original does. Loading fails for a class whose parameters have other names, for example one that assigns its layers to attributes `fc1`, `fc2` and `out` instead of an `nn.Sequential` named `stack`. The error lists the missing and the unexpected names. `eval()` is called

before predicting, as in the PyTorch tutorial on saving models. It switches layers such as dropout to their prediction-time behaviour.

TRAINING ON A GPU

```
device = "cuda" if torch.cuda.is_available() else "cpu"

torch.manual_seed(42)
net = Net(17).to(device)                          # every parameter moved, in place
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
x, y = torch.tensor(xtr, dtype=torch.float32), torch.tensor(ytr, dtype=torch.float32)

for epoch in range(40):
    perm = torch.randperm(len(x))
    for i in range(0, len(x), 128):
        b = perm[i:i + 128]
        xb, yb = x[b].to(device), y[b].to(device)    # each batch to the model's device
        opt.zero_grad()
        loss = loss_fn(net(xb), yb)
        loss.backward()
        opt.step()

with torch.no_grad():
    logits = net(torch.tensor(xte, dtype=torch.float32).to(device))
    pred = (torch.sigmoid(logits) > 0.5).long().cpu().numpy()     # back to main memory
(pred == yte).mean()                              # -> 0.6586   on a GPU, as on the CPU
```

Running the training loop on a GPU takes three changes. The model is moved once with `.to(device)`. This moves its parameters in place and returns the model. Every batch is moved to the same device before the forward pass. Results are copied back to main memory with `.cpu()` before they are converted to NumPy arrays. Choosing the device once, in the first line, lets the same code run where there is no GPU. When the whole training matrix fits in GPU memory, moving `x` and `y` once before the loop saves a copy per batch. The result can differ slightly from a CPU run, because floating-point sums computed in a different order round differently. Here the GPU run and the CPU run both get 1059 of the 1608 test rows right.

FURTHER READING

- The PyTorch basics tutorial, on tensors, autograd, modules and the training loop, in that order.
- BCEWithLogitsLoss, for the loss formula and the `pos_weight` argument.
- torch.optim, for what `step()` does and the arguments Adam takes.
- torch.utils.data, for `Dataset`, `DataLoader`, samplers and `collate_fn`.
- The datasets and dataloaders tutorial.
- Saving and Loading Models, on `state_dict`, `eval()` and the copy required for a best-epoch checkpoint.